MATH 4573: COURSE PROJECT ON RSA CRYPTOGRAPHY

NATALIE CHMURA

1. Introduction

Secret, classified, private, keep-out! Since the scytales of Sparta, to the unbreakable code created by me circa 2008 [1], there has been a long held practice of encoding your messages so they cannot be read by unwanted eyes. Like most of the cool things in life though, we've developed algorithms to help complicate[2] the process. Even cooler? Some of these, like the one we are going to be discussing today, use *primes.* The goal of this paper is to explore the the theory behind how prime numbers and their elementary theorems play into one of the most widely used crypto-systems to this day, as well as testing out our own implementation!

2. Background and Theory

Primarily developed due to the emergance and vulnerability of electronic messaging, the RSA (**R**ivest, **S**hamir, and **A**dleman)[3] algorithm was developed to ensure the safe, private transfer of data. The RSA method is particularly unique in that it offers what's known as a "public key cryptosystem" in which the user is provided an encryption key to encrypt their data with, and using some of the unique characteristics of prime numbers[4] the receiver of the message can decrypt the message with a completely separate key! The encryption and decryption algorithms are represented as so:[RSA]

$$C \equiv E(M) \equiv M^e \pmod{n}, \text{ for a message } M$$
$$D(C) \equiv C^d \pmod{n}, \text{ for a coded message } C$$

As noted in the paper detailing the algorithm, the size itself of the message does not change, as the only encoded values are created in the range from 1 to n - 1. The keys are derived as follows:

Date: April 19, 2024.

[1]**Hint**: it has to do with writing everything backwards, the messages encoded then are still being deciphered by scholars throughout the world to this day!

[2]Complicate as in, develop an intricate series of steps to acheive a goal or purpose. Our purpose? Codes that take a really, really, *reallllly* long time to break

[3]The ones behind the algorithm

[4]And fun methods like Fermat's Little!

(1) Choose two large primes, p and q
(2) Create a "public key modulus", n such that $n = p \cdot q$
(3) Create a "public key", e such that e is relatively prime to $p-1$ and $q-1$
(4) Create a "private key" d such that d is the multiplicative inverse of e, i.e $d \cdot k \equiv 1(\mod e)$, where $k = (p-1) \cdot (q-1) \cdot g$, where $g \in \mathbb{Z}$ such that the condition holds.

Here we can show some simple steps of the derivation as to why this property holds[RSA]:

We can find via. Euler's totiant that our $\phi(n) = \phi(q) \cdot \phi(p)$ following Euler's application of Fermat's Little [NZM91], we know the total can be broken down like so: $(p-1) \cdot (q-1)$

Then, since e is relatively prime to $\phi(n)$, there is a d such that it is the multiplicative inverse: $d \cdot e \equiv 1(\mod \phi(n))$

For full expanded proof, see [RSA] for details.

One might think that knowledge of the public keys would be enough to break the system, and for more trivial cases[5] that may very well be the case. However, given large enough primes that make up n, and due to the relative difficulty of factoring out primes of a large enough number[6], this has proved extremely difficult. The one of the authors of the RSA has graciously provided for us estimates as seen below:

Digits	Number of Operations	Time
50	1.4×10^{10}	3.9 hours
75	9.0×10^{12}	104 days
100	2.3×10^{15}	74 years
200	1.2×10^{23}	3.8×10^{9} years
300	1.5×10^{29}	4.9×10^{15} years
500	1.3×10^{39}	4.2×10^{25} years

TABLE 1. Time it takes to find possible decryption, based on # of digits in n

3. Implementation and Code Snippets

For the following section, I have chosen to implement the RSA encryption algorithm in python, using the steps as detailed above.[7]
The encryption and decryption algorithm steps were relatively straightforward 3, 3 , following the $E(M) \equiv M^e \pmod n$ and $D(C) \equiv C^d \pmod n$ equivalencies, respectively.

[5]Like those generated by my code, as tragically I don't have the power of a super computer that allows me to create primes in the range of 2^{1024} and beyond :(

[6]There is even a cash reward for primes large enough! Check this out!

[7]For actual testing, feel free to check out the attached files! Includes a lovely command prompt UI

```python
def encrypt(message, n, e) :
    '''
        message: string to be encrypted
        n : public moduli key
        e : public moduli key
    '''
    return ((int(message)**int(e)) % n)
```

FIGURE 1. Encryption Algorithm, [rsa_algorithm.py]

```python
def decrypt(message, n, d) :
    '''
        message: string to be decrypted
        n: private moduli key
        d: private moduli key
    '''
    return ((int(message)**int(d)) % n)
```

FIGURE 2. Decryption Algorithm, [rsa_algorithm.py]

For finding the co prime value of n to be used as the public key exponent, e, I chose to generate random integers between the values of 2^{12} and 2^{13} and check their co-primality till I found a suitable match. I wanted the value to be large enough that it would generate a fairly difficult exponent, but small enough that there would be no memory or computation issues.

```python
def find_coprime(p, q) :
    '''
        p: prime 1
        q: prime 2
    '''

    num = (int(p) - 1) * (int(q) - 1)

    e = random.randint((2**12) + 1, (2**13))
    g = math.gcd(num, e)
    while g != 1 :
        e = random.randint((2**12) + 1, (2**13))
        g = math.gcd(num, e)
    return e
```

FIGURE 3. Finding a coprime number, [rsa_algorithm.py]

To find the private key, I originally had intended on using a randomized method as above - this would also allow for diverse encoding and decoding every time the program was

run! However, due to the size of the input and the reliability of the random generation to be, well - random, the approach was changed to be iterative for timing's sake.[8]

```
def find_priv(p, q, e) :
    '''
        p = prime 1
        q = prime 2
        e = public exponent
    '''

    num = (int(p) - 1) * (int(q) - 1)

    find_val = 76;

    d = (((find_val*num) + 1) / e)

    while not float(d).is_integer():
        find_val = find_val + 1
        d = (((find_val*num) + 1) / e)

    return int(d)
```

FIGURE 4. Private Key Generation, [rsa_algorithm.py]

While the above encryption and decryption was performed on simple integers, as indicated by the proof, due to the wonderful world of ASCII and UNICODE we can write much much more as integers or binary code[9]! Binary code can be especially effective when finding the modulus - only should that modulus be a power of two - simply find the rightmost $\log_2(n)$ bits and you have your number! Ex. 100101101010101 00011

4. STEP-BY-STEP

Here we'll walk through a simple case of the operations performed!

- primes $p = 7$, $q = 17$, $\therefore n = 119$
- take e to be 6611, as it is coprime to $(7-1)\cdot(3-1)$
- encrypt our message, 32 as $65^{6611} (\text{mod } 21) = 109$
- if we want to decrypt our message, we can use $d = 59$ to find $109^{59} (\text{mod } 21) = 65$

[8]Originally for both algorithms, I had made the decision to generate numbers within an extreme range - think 100s of digits. However that pushed the calculations to be a bit too large, and even restricting the range down to a more reasonable size led to long >5min wait times, and so for the d calculation I found an iterative approach to be better

[9]01000110 01110101 01101110 00100000 01101000 01101001 01100100 01100100 01100101 01101110 00100000 01101101 01100101 01110011 01110011 01100001 01100111 01100101 00001010

5. Conclusion

In conclusion, we can see how the difficulty in factoring out large numbers into their primes, although simple in theory, can provide a lot of power!

6. Acknowledgments

For this project I would like to my professor, Tyler Genao, for giving me the background necessary to write this paper, as well as an excellent class this past semester. I'd also like to thank the YouTube channel Tom Rocks Maths, who had a very comprehensive video that helped me understand how I want to implement the RSA algorithm. Finally, I'd like to thank the sample project author, "Cole Wittbrodt" as I never used footnotes in my LaTeXdocuments before, and after seeing them used - felt inspired to add a bunch to my own. My documents going forward will no doubt be enhanced by the additional information I can provide!

References

[NZM91] Princeton, NJ (2011). I. Niven, H.S. Zuckerman and H.L. Montgomery, *An introduction to the theory of numbers*, 5th ed., John Wiley & Sons, Inc., New York (1991).

[rsa_algorithm.py] *The code written for this assignment*, Natalie Chmura

[RSA] Massachusetts Institute of Technology (1978) *A Method for Obtaining Digital Signatures and Public-Key Cryptosystems.* Rivest, R. L., Shamir, A., & Adleman, L. https://doi.org/10.21236/ada606588